

**SVKM's Narsee Monjee College of Commerce &
Economics
(AUTONOMOUS)
Juhu Scheme, Vile Parle(West), Mumbai-400 056**

DEPARTMENT OF INFORMATION TECHNOLOGY

Core Java

Title:

Whac-A-Mole Game

45618230001

Stasha Pereira

SYBsc IT-A013

Objectives:

1. The design of an interactive graphical user interface will happen through Java Swing by using a three-by-three grid system for gameplay.
2. A dynamic game system should have randomized mole dynamics, scoring features, real-time timer functionality and difficulty adaptation for the game.
3. A Leaderboard system should be integrated to keep track of best player scores yet allow users to reset the high scores.
4. The user experience gets improved by both instant user feedback and integrated game control elements.
5. The code base achieves maximum efficiency during execution through both object-oriented programming principles and effective event handling improvements.

This project showcases *Java-based game development* with an engaging and dynamic arcade experience.

Abstract:

The program based on the ***Whac-A-Mole*** game features a *Java Swing* implementation that includes a *3x3 grid-based game board* with an interactive graphical user interface (GUI). By design the game demands users click moles when they appear and stay away from plants which serve as obstacles. The gameplay requires reaching the highest score during a 60-second countdown as the game automatically speeds up both moles and plants according to player progress.

The application features a leaderboard mechanism which saves player data in *scores.txt* files for showing the top scores. The game shows *real-time feedback* through score display, miss counter and a minute-timer which improves user interaction. Game players can benefit from the restart function and the leaderboard reset feature for playing the game repeatedly. Through *randomized* plant and mole location selection and the difficulty level adjustment the game maintains its dynamic entertainment value.

All fundamental programming concepts such as *event-driven programming* and *multithreading through Swing Timers combined with file handling for persistent storage and object-oriented design* form the base of this project. This game combines educational value with entertainment since it harnesses Java GUI capabilities to develop interactive gaming features.

Code:

```
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.nio.file.*;
import java.util.*;
import javax.swing.*;

public class WhacAMole
{
    int boardWidth = 600;
    int boardHeight = 800;

    JFrame frame = new JFrame("Whac A Mole");
    JLabel textLabel = new JLabel();
    JLabel missCountLabel = new JLabel();
    JLabel timerLabel = new JLabel();
    JPanel textPanel = new JPanel();
    JPanel boardPanel = new JPanel();
    JPanel controlPanel = new JPanel();
    JButton[] board = new JButton[9];
    ImageIcon moleIcon;
    ImageIcon plantIcon;

    JButton currMoleTile;
    JButton[] currPlantTiles = new JButton[3];

    Random random = new Random();
    javax.swing.Timer setMoleTimer;
    javax.swing.Timer setPlantTimer;
    javax.swing.Timer countdownTimer;
    int score = 0;
    int timeLeft = 60;
    String playerName;
    int missCount = 0;
    int moleDelay = 800;
    int plantDelay = 1200;

    WhacAMole()
    {
        playerName = getPlayerName();
        frame.setSize(boardWidth, boardHeight);
        frame.setLocationRelativeTo(null);
        frame.setResizable(false);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLayout(new BorderLayout());

        textLabel.setFont(new Font("Arial", Font.PLAIN, 50));
        textLabel.setHorizontalAlignment(JLabel.CENTER);
```

```
textLabel.setText("Score: " + score);
textLabel.setOpaque(true);

missCountLabel.setFont(new Font("Arial", Font.PLAIN, 30));
missCountLabel.setHorizontalAlignment(JLabel.CENTER);
missCountLabel.setText("Misses: " + missCount);
missCountLabel.setOpaque(true);

timerLabel.setFont(new Font("Arial", Font.PLAIN, 30));
timerLabel.setHorizontalAlignment(JLabel.CENTER);
timerLabel.setText("Time Left: " + timeLeft);
timerLabel.setOpaque(true);

textPanel.setLayout(new BorderLayout());
textPanel.add(textLabel, BorderLayout.NORTH);
textPanel.add(missCountLabel, BorderLayout.CENTER);
textPanel.add(timerLabel, BorderLayout.SOUTH);
frame.add(textPanel, BorderLayout.NORTH);

boardPanel.setLayout(new GridLayout(3, 3));
frame.add(boardPanel);

Image plantImg = new ImageIcon(getClass().getResource("./piranha.png")).getImage();
plantIcon = new ImageIcon(plantImg.getScaledInstance(150, 150,
java.awt.Image.SCALE_SMOOTH));

Image moleImg = new ImageIcon(getClass().getResource("./monty.png")).getImage();
moleIcon = new ImageIcon(moleImg.getScaledInstance(150, 150,
java.awt.Image.SCALE_SMOOTH));

Color lightBrown = new Color(222, 184, 135);

for (int i = 0; i < 9; i++)
{
    JButton tile = new JButton();
    tile.setBackground(lightBrown);
    tile.setOpaque(true);
    tile.setBorderPainted(false);
    board[i] = tile;
    boardPanel.add(tile);
    tile.setFocusable(false);

    tile.addActionListener(new ActionListener()
    {
        public void actionPerformed(ActionEvent e)
        {
            JButton tile = (JButton) e.getSource();
            if (tile == currMoleTile)
            {
                score += 10;
                textLabel.setText("Score: " + score);
            }
        }
    });
}
```

```

        adjustSpeed();
    }
    else
    {
        boolean isPlantHit = false;
        for (JButton plantTile : currPlantTiles)
        {
            if (tile == plantTile)
            {
                isPlantHit = true;
                break;
            }
        }
        if (isPlantHit || tile.getIcon() == null)
        {
            missCount++;
            missCountLabel.setText("Misses: " + missCount);

            if (missCount >= 5)
            {
                gameOver();
            }
        }
    }
}

});

}

setMoleTimer = new javax.swing.Timer(moleDelay, new ActionListener()
{
    public void actionPerformed(ActionEvent e)
    {
        if (currMoleTile != null)
        {
            currMoleTile.setIcon(null);
            currMoleTile = null;
        }

        int num = random.nextInt(9);
        JButton tile = board[num];
        if (isTileOccupied(tile)) return;

        currMoleTile = tile;
        currMoleTile.setIcon(moleIcon);
    }
});

setPlantTimer = new javax.swing.Timer(plantDelay, new ActionListener()
{
    public void actionPerformed(ActionEvent e)
    {
        for (int i = 0; i < currPlantTiles.length; i++)

```

```
{
    if (currPlantTiles[i] != null)
    {
        currPlantTiles[i].setIcon(null);
        currPlantTiles[i] = null;
    }
    int num = random.nextInt(9);
    JButton tile = board[num];
    if (isTileOccupied(tile))
    {
        i--;
        continue;
    }
    currPlantTiles[i] = tile;
    currPlantTiles[i].setIcon(plantIcon);
}

});

JButton restartButton = new JButton("Restart Game");
restartButton.addActionListener(new ActionListener()
{
    public void actionPerformed(ActionEvent e)
    {
        resetGame();
    }
});

JButton clearLeaderboardButton = new JButton("Clear Leaderboard");
clearLeaderboardButton.addActionListener(new ActionListener()
{
    public void actionPerformed(ActionEvent e)
    {
        clearLeaderboard();
    }
});

controlPanel.add(restartButton);
controlPanel.add(clearLeaderboardButton);
frame.add(controlPanel, BorderLayout.SOUTH);

startGame();
frame.setVisible(true);
}

private String getPlayerName()
{
    return JOptionPane.showInputDialog("Enter your name:");
}

private void startGame()
{

```

```
score = 0;
missCount = 0;
timeLeft = 60;
textLabel.setText("Score: " + score);
missCountLabel.setText("Misses: " + missCount);
timerLabel.setText("Time Left: " + timeLeft);
moleDelay = 800;
plantDelay = 1200;
setMoleTimer.setDelay(moleDelay);
setPlantTimer.setDelay(plantDelay);

countdownTimer = new javax.swing.Timer(1000, new ActionListener()
{
    public void actionPerformed(ActionEvent e)
    {
        timeLeft--;
        timerLabel.setText("Time Left: " + timeLeft);
        if (timeLeft <= 0) {
            countdownTimer.stop();
            gameOver();
        }
    }
});
countdownTimer.start();
setMoleTimer.start();
setPlantTimer.start();
}

private boolean isTileOccupied(JButton tile)
{
    return tile == currMoleTile || Arrays.asList(currPlantTiles).contains(tile);
}

private void resetGame()
{
    for (JButton tile : board)
    {
        tile.setIcon(null);
        tile.setEnabled(true);
    }
    startGame();
}

private void adjustSpeed()
{
    if (score % 30 == 0 && moleDelay > 200)
    {
        moleDelay -= 100;
        plantDelay -= 100;
        setMoleTimer.setDelay(moleDelay);
        setPlantTimer.setDelay(plantDelay);
    }
}
```

```
    }  
}  
  
private void gameOver()  
{  
    textLabel.setText("Game Over: " + score);  
    setMoleTimer.stop();  
    setPlantTimer.stop();  
    countdownTimer.stop();  
    for (JButton tile : board)  
    {  
        tile.setEnabled(false);  
    }  
    logScore(score);  
    displayLeaderboard();  
}  
  
private void logScore(int score)  
{  
    if (playerName != null && !playerName.trim().isEmpty())  
    {  
        try (BufferedWriter writer = new BufferedWriter(new FileWriter("scores.txt",  
true)))  
        {  
            writer.write(score + ": " + playerName + " (" + new Date() + ")");  
            writer.newLine();  
        } catch (IOException e)  
        {  
            e.printStackTrace();  
        }  
    }  
}  
  
private void displayLeaderboard()  
{  
    java.util.List<String> scores = new ArrayList<>();  
    try  
    {  
        scores = Files.readAllLines(Paths.get("scores.txt"));  
    } catch (IOException e)  
    {  
        e.printStackTrace();  
    }  
  
    StringBuilder leaderboard = new StringBuilder("Leaderboard:\n");  
    Collections.sort(scores, (a, b) -> {  
        int scoreA = Integer.parseInt(a.split(": ")[0]);  
        int scoreB = Integer.parseInt(b.split(": ")[0]);  
        return Integer.compare(scoreB, scoreA);  
    });  
  
    for (int i = 0; i < Math.min(scores.size(), 10); i++)
```

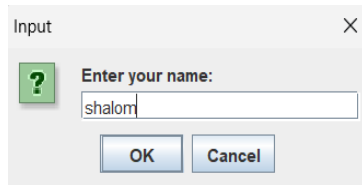
```
{
    leaderboard.append(scores.get(i)).append("\n");
}

JOptionPane.showMessageDialog(frame, leaderboard.toString(), "Leaderboard",
JOptionPane.INFORMATION_MESSAGE);
}

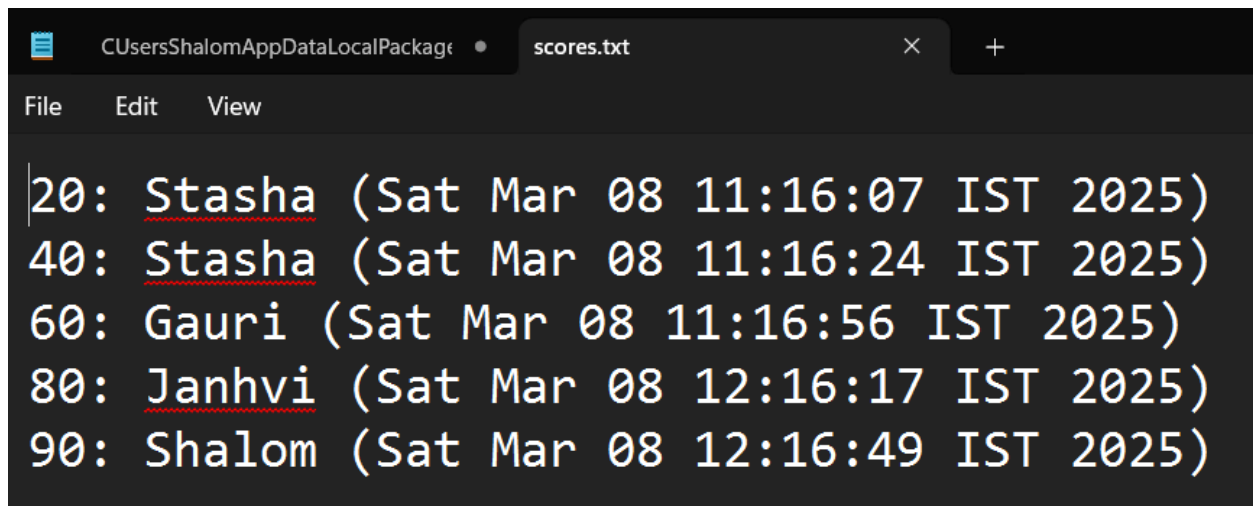
private void clearLeaderboard()
{
    try {

        Files.newBufferedWriter(Paths.get("scores.txt"),
StandardOpenOption.TRUNCATE_EXISTING).close();
        JOptionPane.showMessageDialog(frame, "Leaderboard cleared!", "Success",
JOptionPane.INFORMATION_MESSAGE);
    } catch (IOException e)
    {
        e.printStackTrace();
        JOptionPane.showMessageDialog(frame, "Error clearing leaderboard.", "Error",
JOptionPane.ERROR_MESSAGE);
    }
}

public static void main(String[] args)
{
    SwingUtilities.invokeLater(() -> new WhacAMole());
}
}
```

Output:

Input dialog box titled "Input" with a close button (X). It contains a green question mark icon, the text "Enter your name:", a text input field containing "shalom", and "OK" and "Cancel" buttons.



File Explorer window showing the contents of "scores.txt" in the directory "CUsersShalomAppDataLocalPackage". The file contains a list of scores and timestamps:

```
20: Stasha (Sat Mar 08 11:16:07 IST 2025)
40: Stasha (Sat Mar 08 11:16:24 IST 2025)
60: Gauri (Sat Mar 08 11:16:56 IST 2025)
80: Janhvi (Sat Mar 08 12:16:17 IST 2025)
90: Shalom (Sat Mar 08 12:16:49 IST 2025)
```

